



University of Puerto Rico
Mayaguez Campus
Department of Electrical and Computer Engineering



Agents - Chapter 2

Luis A. Hernandez Padilla
Andrés E. Muñoz Ríos
Sofia A. Chamorro Messa
Sergio Paulo Ruben Avila Concepción

Prof. José Fernando Vega Riveros
ICOM5015-001D
March 06, 2026

Purpose of the Assignment The fundamental purpose is to design, implement and evaluate a multi-agent system in the Vacuum Cleaner World environment, taking into account the principles of modern Artificial Intelligence proposed by Russell and Norvig; with the central objective of demonstrating the importance of a modular software architecture that strictly separates the responsibilities of the Agent, the Environment and the Performance Measure.

Key Hypothesis “It is postulated that the efficiency of an intelligent agent is directly proportional to the sophistication of its internal structure and its alignment with the performance measure (P). [1] Specifically: A Simple Reflex Agent will significantly outperform a Random Agent by replacing randomness with a deterministic condition-action policy, minimizing redundant actions in fully observable environments. [1][3] However, in environments that penalize unnecessary movement, a Model-Based Agent (with State) will outperform the Simple Reflex Agent. [1] This is because its ability to maintain an internal representation of the world (memory) allows it to identify task completion and execute neutral actions (NoOp), thus avoiding cumulative penalties for redundant exploration.” [2][3]

Concepts PEAS Framework: Formalization methodology that defines success (Performance), the physical context (Environment), the action capabilities (Actuators), and the information input (Sensors). [1]

Rationality: An optimization criterion where the agent selects the action that maximizes expected performance, subject to its perceptual and knowledge limitations. [1]

Simple Reflex Agent: A reactive architecture that maps current perceptions directly to actions using predefined rules, without considering past history. [1][2]

Random Agent: A stochastic decision model used as an experimental control to validate the performance gain of rational agents. [3]

Model-Based Agent (with State): An architecture that incorporates internal memory to track non-visible aspects of the environment or past states, allowing for more informed decision-making. [1]

Transition Dynamics: A mathematical or logical function that describes the change in the state of the environment as a consequence of a specific agent action. [1][2]

Architectural Modularity: An engineering principle that ensures independence between the agent program and the simulator, facilitating scalability and component interchangeability. [2]

Information gathered to complete the task For the successful execution of this project, the following information was compiled and analyzed:

Bibliographic Reference: Chapters 2 and 3 of "Artificial Intelligence: A Modern Approach" (Russell & Norvig, 3rd Edition), specifically exercises 2.8 and 2.11, were consulted to understand the logic of the vacuum cleaner world.

UC Berkeley Code Repository: The class structure of Berkeley's AI projects (AIMA-Python) was analyzed to ensure

that the implementation was compatible with Python academic standards.

Environment *Technical Specifications:*

Structure: A world with two locations (A and B).

Actions: The finite set of allowed commands (Left, Right, Suck, NoOp).

Performance Metric: The definition of the reward function (+1 per clean frame per time step). *Simulation Data:* Results were collected from 100 independent trials for each agent, ensuring that performance comparisons had statistical validity and were not dependent on a single initial setup.

Experimental Design Choice

To answer whether a randomized reflex agent can outperform a simple reflex agent, three agent types were designed and compared: a Random Agent, a Simple Reflex Agent, and a Reflective Random Agent. The Reflective Random Agent was specifically designed by preserving the core reflex rule (suck when dirty) while replacing the deterministic movement decision with a randomized selection among Left, Right, and NoOp. This isolation of a single variable ensures that any performance difference is directly attributable to the addition of randomness. All agents were tested in the same two-room vacuum world under identical conditions.

Measurement Methodology

```
def execute_action(self, action):
    """Lógica de transición correcta y cálculo de rendimiento (Criterio 2 y 3)"""
    if action == 'Aspirar':
        if self.dirt_status[self.agent_location] == 'Sucio':
            self.dirt_status[self.agent_location] = 'Limpio'
            self.performance_score += 10 # Recompensa por limpiar

    elif action == 'Derecha':
        idx = self.locations.index(self.agent_location)
        if idx < len(self.locations) - 1:
            self.agent_location = self.locations[idx + 1]
            self.performance_score -= 1 # Penalización por moverse

    elif action == 'Izquierda':
        idx = self.locations.index(self.agent_location)
        if idx > 0:
            self.agent_location = self.locations[idx - 1]
            self.performance_score -= 1 # Penalización por moverse

    elif action == 'NoOp':
        pass # No penaliza ni recompensa
```

Figure 1. How actions were rewarded.

Performance was measured using a reward function of +1 point for every clean room per time step. To ensure statistical validity, 100 independent trials were run per agent, with randomized initial conditions across trials. Results were averaged across all trials to produce a final performance score for each agent, eliminating dependency on any single favorable or unfavorable starting configuration.

Results and Analysis

Two experiments were conducted, comparing the efficiency of the agents under the established performance metrics (P):

Exercise 2.11 (Reactive vs. Random): The Reactive Agent (8.45) outperforms the Random Agent (2.92) by almost three times, confirming that a policy based on condition-action rules ("if it's dirty, vacuum") is fundamental to rationality in this environment. The random agent wastes too many actions on unnecessary movements or by vacuuming where it is already clean.

--- Resultados tras 100 ensayos ---
Puntuación Promedio Agente Reactivo: 0.29
Puntuación Promedio Agente Reflexivo Aleatorio: 10.36

Figure 2. Results Reactive vs. Reflective

Exercise 2.14 - Advanced Comparisons:

Question 2 (Reactive vs. Reflective Random): Here we see a curious result: the Reflective Random Agent (10.36) drastically outperforms the Reactive Agent (0.29). This suggests that in this specific scenario, the reactive agent might be "trapped" in an inefficient loop, or that the metric severely penalizes constant movement, while randomness allows for better exploration of the space.

Puntuación Promedio Agente Aleatorio: -5.78
Puntuación Promedio Agente Reactivo: 7.0
Puntuación Promedio Agente Reflexivo Aleatorio: -1.45

Figure 3. Results for Hostile Environment.

Question 3 (Hostile Environment): In an environment designed to penalize randomness, the Random Agent (-5.78) and the Random Reflective Agent (-1.45) score negatively. The Reactive Agent (7.0) demonstrates its robustness, maintaining positive performance by following logical rules that prevent penalized actions.

--- Resultados tras 100 ensayos ---
Puntuación Promedio Agente Reactivo: 1.96
Puntuación Promedio Agente Reflexivo con Estado: 13.67

Figure 3. Results for State vs. Simple Reflex.

Question 4 (State vs. Simple Reflex): This is the most conclusive result. The State Agent (13.67) significantly outperforms the Reactive Agent (1.96). This is because the state agent possesses an internal model (memory) that allows it to know which squares it has already cleared, avoiding redundant movements and optimizing its energy.

Conclusions and lessons learned

In this work, a modular vacuum cleaner world simulator was implemented following the PEAS framework.

The architecture clearly separates the environment, the agent program, and the performance evaluator, allowing for the testing of different agent strategies.

The experiments showed that a simple reflex agent achieves significantly higher performance than a random agent, confirming the importance of using perceptual information for rational decision-making.

This simulator can be easily extended to include larger environments, imperfect sensors, or more sophisticated agents, making it a useful platform for experimenting with different intelligent agent paradigms.

The Superiority of the Internal Model: The data demonstrate that the ability to maintain an internal state (Stateful Agent) is the most successful strategy. By "remembering" the world, the agent ceases to be purely reactive and begins to act with an efficiency approaching total optimization.

Impact of Environment Design: The results of Question 3 prove that an agent's rationality is relative to its environment. An agent that is successful in a simple world can fail spectacularly if the environment introduces penalties for random actions or unnecessary movements.

Limitations of Simple Reflex: Although the

reactive agent is better than the random one under normal conditions, its lack of memory makes it vulnerable in environments where current perceptions are insufficient to make the best long-term decision.

• Lessons Learned

We learned that how we define success (positive points for cleaning vs. negative points for moving) determines how "intelligent" an agent will appear. A change in the metric can make a random agent appear better than a reactive one.

The separation of components allowed us to test different "brains" (agents) in the same "body" (actuators/sensors) and "world" (environment) without rewriting all the code. This is the basis of scientific experimentation in AI.

An agent can be rational, but if its sensors (S) are limited or if it lacks a model, it will perform poorly in processing the history of its perceptions.

In AI, memory is not just about "storing data," but about building a model of the world that allows predicting which actions are unnecessary, saving resources and maximizing performance.

TASKS

1. **Source Code in Python:** Andrés,
2. **Purpose of the assignment:** Sofia
3. **Key hypothesis:** Sofia
4. **Experimental design choices:** Andrés,
5. **Measurement methodology:**
6. **Results and analysis:** Sofia
7. **Interpretation:**
8. **Conclusión:** Sofia

Bibliography

- [1] Russell, S. J., & Norvig, P. (2010). Artificial Intelligence: A Modern Approach (3rd ed.). Upper Saddle River, NJ: Prentice Hall. (Capítulos 2 y 3; Ejercicios 2.8 y 2.11).
- [2] UC Berkeley AI Materials. AIMA-Python Repository. Recovered de <https://github.com/aimacode/aima-python>.
- [3] Task Environment Technical Specifications (PEAS). Based on course requirements and the Russell & Norvig problems.
 - Actuators: Command set (Left, Right, Suck, NoOp).
 - Performance Metric: Reward function (+1 per clean frame per step).
- [4] Experimental Validation Protocol. Data collection methodology using 100 independent trials to ensure statistical validity and neutrality of the initial configuration.

